

SMART IMAGE ANALYZER

A Computer Vision Based Image Processing and Feature Analysis System

Computer Vision Project Report

Student Name	Jaiswal Aryan Santosh
Registration Number	24BAI10482
Course / Section	Computer Vision
Faculty / Supervisor	Dr. Prakash N B
Submission Date	18 September 2026

Technologies: Python, OpenCV, NumPy, Streamlit, Pillow, Pytest

1. Introduction

Smart Image Analyzer is an interactive Computer Vision application designed to analyze an uploaded image through a sequence of image-processing operations. The system converts an input image into useful representations, detects edges, analyzes external contours and extracts statistical and edge-based features.

The application is implemented using a modular Python architecture. Individual Computer Vision operations are separated into reusable modules, while Streamlit provides the user interface. Automated unit tests are included to validate core processing functions.

2. Problem Statement

Basic image-processing operations are often implemented as separate scripts, making it difficult for a learner or user to view the complete analysis workflow in one place. The project addresses this by providing a single interactive application that accepts an image and presents preprocessing, edge detection, contour analysis, feature extraction, and visualization results.

3. Objectives

- Process and analyze digital images using Computer Vision techniques.
- Convert images into grayscale and binary representations.
- Detect image edges using the Canny edge detection algorithm.
- Detect and analyze external object contours.
- Extract statistical and edge-based image features.
- Provide an interactive interface for viewing processing results.
- Maintain a modular and testable codebase.

4. Requirements

4.1 Functional Requirements

FR1 – Image Upload: The user can upload JPG, JPEG, or PNG images.

FR2 – Image Preprocessing: The system performs grayscale conversion, Gaussian blur, and binary thresholding.

FR3 – Edge Detection: The system detects image edges using Canny edge detection.

FR4 – Contour and Shape Analysis: The system detects external contours and calculates area, perimeter, and bounding-box dimensions.

FR5 – Feature Extraction: The system calculates image statistics and edge features.

FR6 – Result Visualization: The processed outputs and analysis summary are shown through the Streamlit interface.

4.2 Non-Functional Requirements

- Usability – The interface should be simple and understandable.
- Performance – Image operations should execute interactively for normal image sizes.
- Reliability – Invalid or unreadable images should be handled safely.
- Maintainability – Computer Vision operations are separated into modular files.
- Portability – The application can be run in a Python virtual environment.

5. System Architecture

USER	
↓	
Streamlit UI / Image Upload	
↓	
Image Preprocessing	Grayscale • Gaussian Blur • Thresholding
↓	
Computer Vision Processing	Canny Edge Detection • Contour Analysis
↓	
Feature Extraction	Image Statistics • Edge Features
↓	
Results & Visualization	

Figure 1: System Architecture

6. Application Workflow

The application follows a sequential workflow: the user uploads an image, the input is validated, preprocessing is performed, edges are detected, contours are analyzed, features are extracted, and the final results are displayed.

Start	
↓	
Upload Image	
↓	
Validate Image	
↓	
Preprocess Image	Grayscale / Blur / Threshold
↓	
Detect Edges	Canny
↓	
Analyze Contours	Area / Perimeter / Bounding Box
↓	
Extract Features	Statistics / Edge Features
↓	

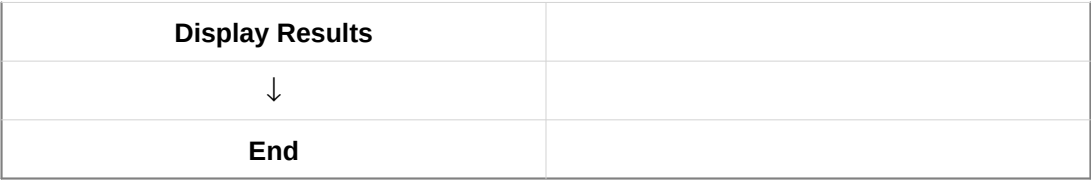


Figure 2: Application Workflow

7. Use Case Diagram

The primary actor is the User. The User uploads an image and views the outputs generated by the system, including preprocessing results, edge detection, contour analysis, extracted features, and the final analysis summary.

USER	SMART IMAGE ANALYZER
User	Upload Image
	View Preprocessing Results
	View Edge Detection
	View Contour Analysis
	View Extracted Features
	View Analysis Summary

Figure 3: Use Case Diagram

8. Module Structure

The project uses separate Python modules for preprocessing, edge detection, contour analysis, feature extraction, and visualization. The Streamlit application coordinates these modules, while Pytest tests validate core functionality.

File / Module	Responsibility
app.py	Streamlit interface and workflow orchestration
preprocessing.py	Grayscale, Gaussian blur, thresholding
edge_detection.py	Canny edge detection
contour_analysis.py	Contour detection, measurements, visualization
feature_extraction.py	Image statistics and edge features
visualization.py	BGR-to-RGB conversion for display
tests/test_processing.py	Automated unit tests

Figure 4: Module Structure / Module Responsibilities

9. Implementation / Methodology

9.1 Image Preprocessing

The uploaded BGR image is converted to grayscale using OpenCV. A Gaussian blur with a 5×5 kernel is then applied to reduce image noise. The blurred grayscale image is converted into a binary image using a fixed threshold value of 127.

9.2 Edge Detection

The Canny edge detector is applied to the grayscale image using lower and upper thresholds of 50 and 150. The resulting binary edge map highlights structural boundaries in the image.

9.3 Contour and Shape Analysis

External contours are detected from the thresholded image. For each contour, the system calculates area, perimeter, and bounding-box width and height. Contours are also drawn on a copy of the original image for visualization.

9.4 Feature Extraction

Image statistics include width, height, number of channels, mean intensity, standard deviation, minimum intensity, and maximum intensity. Edge features include the number of detected edge pixels and edge density.

10. Results

The implemented application successfully processed a 1920×1080, 3-channel sample image. The sample run produced 3,388 external contours and 465,215 edge pixels, with an edge density of 0.2244.

Metric	Observed Value
Image dimensions	1920 × 1080
Channels	3
Mean intensity	102.90
Standard deviation	51.35
Minimum intensity	2
Maximum intensity	244
External contours	3,388
Edge pixels	465,215
Edge density	0.2244

Smart Image Analyzer

A Computer Vision application for image preprocessing, edge detection, contour analysis, and feature extraction.

Upload an image



1. Image Preprocessing



Original Image



Grayscale Image



Thresholded Image

Figure 5: Image Preprocessing Results

2. Edge Detection



Figure 6: Canny Edge Detection Result

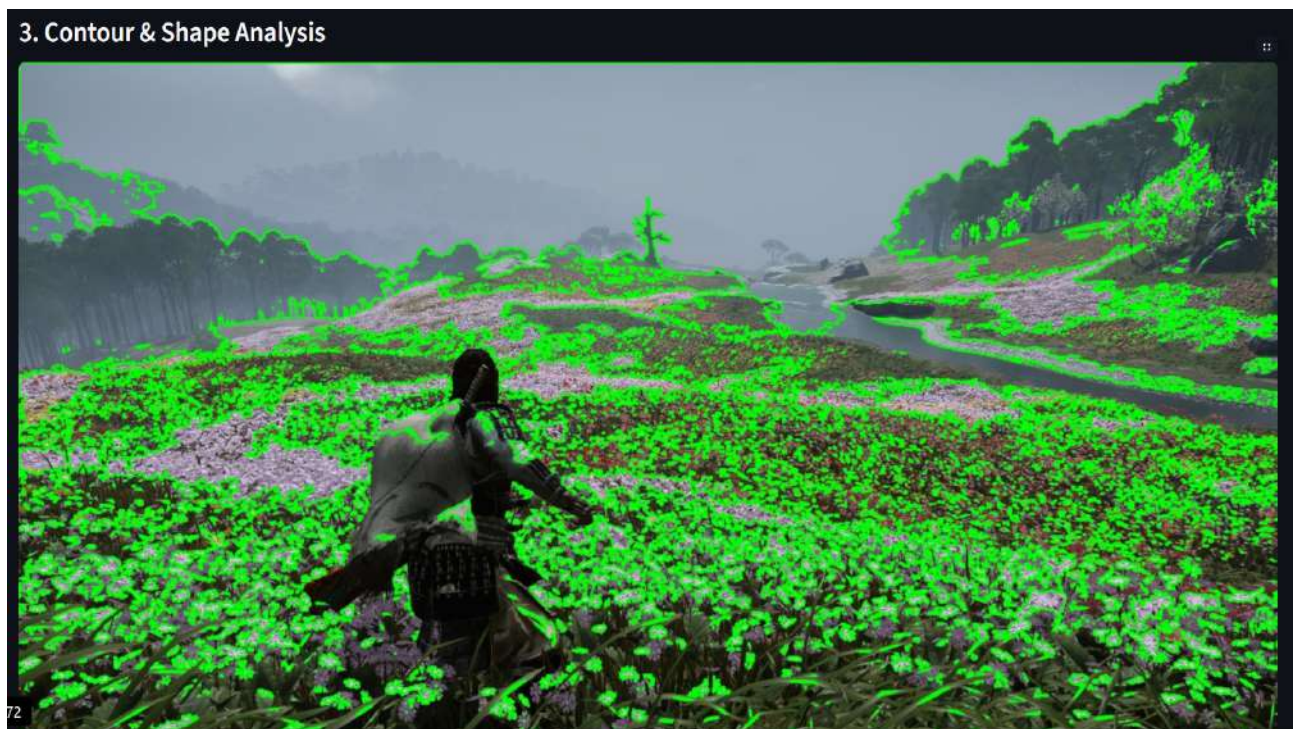


Figure 7: Contour and Shape Analysis Result

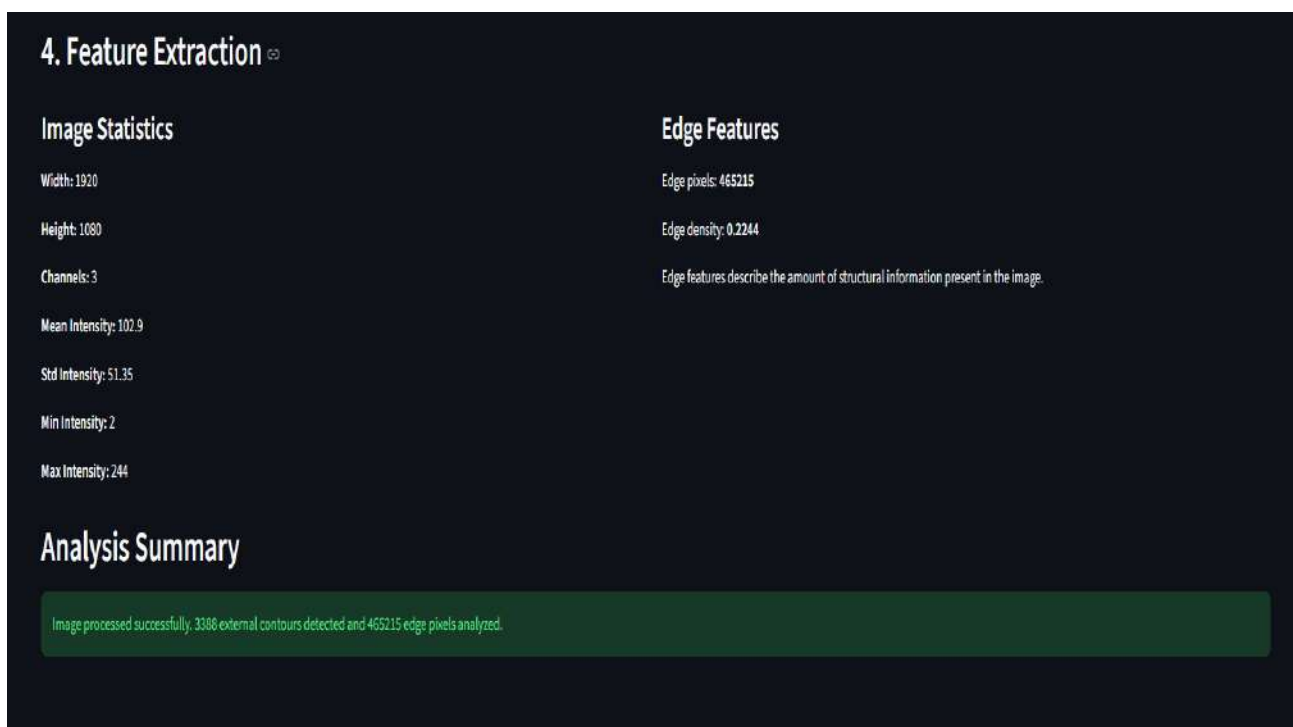


Figure 8: Feature Extraction and Analysis Summary

11. Testing

Automated unit testing was implemented using Pytest. A synthetic 100×100 RGB test image was used to validate core image-processing functions.

Test	Result
Grayscale conversion	PASSED
Gaussian blur	PASSED
Thresholding	PASSED
Canny edge detection	PASSED
Image statistics	PASSED
Edge feature extraction	PASSED
Total	6 / 6 PASSED

12. Error Handling and Validation

The application checks whether the uploaded image can be decoded before processing. Supported upload types are JPG, JPEG, and PNG. If an unreadable image is encountered, an error message is displayed and processing is stopped. The modular design also makes individual processing functions independently testable.

13. Challenges Faced

The project required coordinating multiple image-processing stages while keeping the implementation modular. Handling OpenCV's BGR image representation together with Streamlit's RGB display requirements was also addressed through a dedicated visualization utility. Automated tests were added to verify the core processing pipeline.

14. Learning Outcomes

The project provided practical experience with grayscale conversion, Gaussian filtering, thresholding, Canny edge detection, contour analysis, image statistics, feature extraction, Streamlit interfaces, modular Python development, and unit testing with Pytest. It also demonstrated how individual Computer Vision operations can be combined into an end-to-end analysis workflow.

15. Innovation / Depth

The project combines several fundamental Computer Vision operations in one interactive application rather than implementing them as isolated examples. It additionally calculates quantitative contour and edge features and presents them alongside visual processing outputs. The modular architecture and automated tests support extension and reuse.

16. Future Scope

Possible extensions include object detection using deep learning, face detection and recognition, advanced feature descriptors such as SIFT and HOG, image classification, batch image processing, configurable thresholds, and downloadable analysis reports.

17. Conclusion

Smart Image Analyzer successfully demonstrates an end-to-end Computer Vision workflow for image preprocessing, edge detection, contour analysis, and feature extraction. The working Streamlit application, modular implementation, validation logic, automated tests, and documented architecture provide a complete project implementation suitable for demonstrating fundamental Computer Vision concepts.

18. References

1. OpenCV Documentation – Image Processing and Computer Vision functions.
2. NumPy Documentation – Numerical and array-based image analysis operations.
3. Streamlit Documentation – Interactive Python application framework.
4. Pillow Documentation – Python Imaging Library.
5. Pytest Documentation – Python testing framework.

Project Repository

<https://github.com/ARYAN-JAISWAL7425/Image-Analyzer>